

Lab Program 3

Title: Integrate Git with Docker for source-controlled Applications Builds

STEP 1: Create a directory and navigate inside it

```
BitterSteel@victus:~$ cd Desktop/  
BitterSteel@victus:~/Desktop$ mkdir lab4  
BitterSteel@victus:~/Desktop$ cd lab4/
```

STEP 2: Create **package.json** file

\$ npm init -y

```
BitterSteel@victus:~/Desktop/lab4$ npm init  
This utility will walk you through creating a package.json file.  
It only covers the most common items, and tries to guess sensible defaults.  
  
See `npm help init` for definitive documentation on these fields  
and exactly what they do.  
  
Use `npm install <pkg>` afterwards to install a package and  
save it as a dependency in the package.json file.  
  
Press ^C at any time to quit.  
package name: (lab4)  
version: (1.0.0)  
description:  
entry point: (index.js)  
test command:  
git repository:  
keywords:  
author:  
license: (ISC)  
About to write to /home/BitterSteel/Desktop/lab4/package.json:  
  
{  
  "name": "lab4",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "author": "",  
  "license": "ISC"  
}
```

STEP 3: Install dependencies

\$ npm install express

```
BitterSteel@victus:~/Desktop/lab4$ npm install express
added 68 packages, and audited 69 packages in 10s

18 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

STEP 4: Create **index.js** file

\$ nano index.js

Write the node application

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send("Docker + Github integration");
});

app.listen(port, () => {
  console.log(`server is running at port ${port}`);
});
```

STEP 5: Write **Dockerfile**

\$ nano Dockerfile

```
FROM node:18 AS build

WORKDIR /app

COPY package*.json ./

RUN npm init -y

COPY . .

FROM node:18-slim

WORKDIR /app

COPY --from=build /app /app

EXPOSE 3000

CMD ['node', 'index.js']
```

Project Structure

lab4/

node_modules

index.js

Dockerfile

package-lock.json

package.json

STEP 6: Initialise directory with git

\$ git init

```
BitterSteel@victus:~/Desktop/lab4$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/BitterSteel/Desktop/lab4/.git/
```

STEP 7: Stage the files

\$ git add .

```
BitterSteel@victus:~/Desktop/lab4$ git add .
```

STEP 8: Commit the files

\$ git commit -m "message"

```
BitterSteel@victus:~/Desktop/lab4$ git commit -m "update folder"
[master 00ac514] update folder
```

STEP 9: Set the branch to main

\$ git branch -M main

```
BitterSteel@victus:~/Desktop/lab4$ git branch -M main
```

STEP 10: Create a new repository in Github

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1

General

Owner *

 kurniajay

Repository name *

lab4

✔ lab4 is available.

Great repository names are short and memorable. How about [fantastic-palm-tree?](#)

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

STEP 11: Generate Personal Access Token to authenticate git operations with Github

(Store the PAT somewhere safely to use it later)

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

lab4

What's this token for?

Expiration

Select date *

Custom ▾

22 / 11 / 2025 📅

The token will expire on the selected date

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events
☐ workflow	Update GitHub Action workflows
☐ write:packages	Upload packages to GitHub Package Registry
☒ read:packages	Download packages from GitHub Package Registry
☒ delete:packages	Delete packages from GitHub Package Registry
☒ admin:org	Full control of orgs and teams, read and write org projects
☒ write:org	Read and write org and team membership, read and write org projects

STEP 12: Link the github repo to the project folder

```
$ git remote add origin https://github.com/kurniajay/lab4.git
```

```
BitterSteel@victus:~/Desktop/lab4$ git remote add origin https://github.com/kurniajay/lab4.git
```

STEP 13: Push to the repository in github

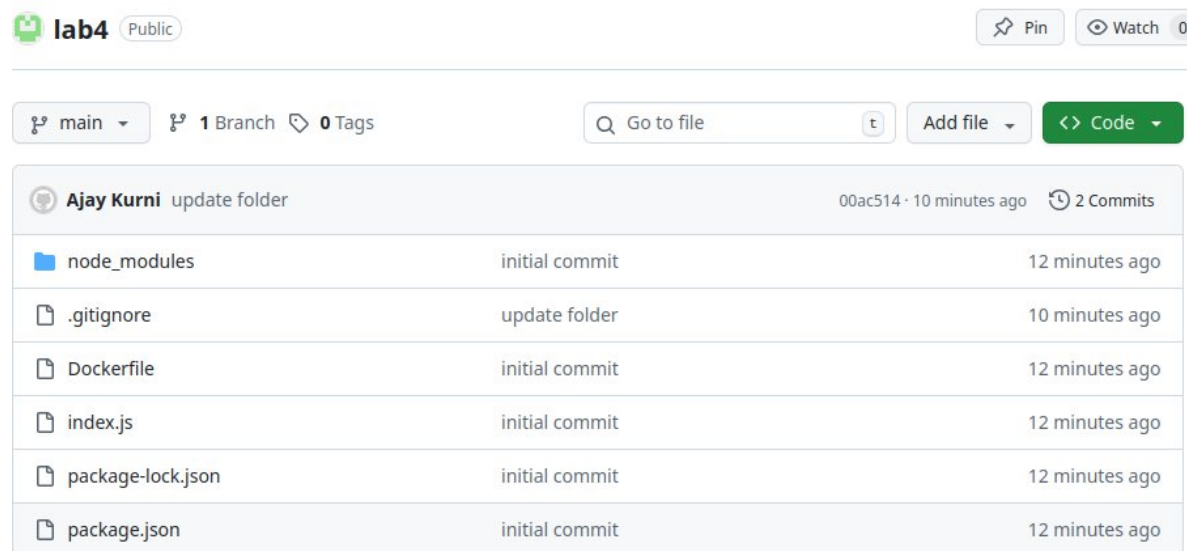
It asks

- Username : kurniajay

-Password : Paste generated PAT

```
BitterSteel@victus:~/Desktop/lab4$ git push -u origin main
Username for 'https://github.com': kurniajay
Password for 'https://kurniajay@github.com':
Enumerating objects: 722, done.
Counting objects: 100% (722/722), done.
Delta compression using up to 12 threads
Compressing objects: 100% (668/668), done.
Writing objects: 100% (722/722), 725.66 KiB | 7.40 MiB/s, done.
Total 722 (delta 143), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (143/143), done.
To https://github.com/kurniajay/lab4.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

STEP 14: Check the repository in github if the push is successfull



The screenshot shows the GitHub interface for a repository named 'lab4' which is public. The repository is owned by 'Ajay Kurni' and has 2 commits. The main branch is selected. Below the repository header, there is a table of files and folders. The files listed are 'node_modules', '.gitignore', 'Dockerfile', 'index.js', 'package-lock.json', and 'package.json'. Each file has a commit message and a timestamp indicating when it was last updated.

File/Folder	Commit Message	Timestamp
node_modules	initial commit	12 minutes ago
.gitignore	update folder	10 minutes ago
Dockerfile	initial commit	12 minutes ago
index.js	initial commit	12 minutes ago
package-lock.json	initial commit	12 minutes ago
package.json	initial commit	12 minutes ago